

DriverApp - Complete Code & Architecture Documentation

For: Client Technical Review
Version: 1.0.0
Last Updated: March 2026
Platform: Flutter (iOS + Android)
Backend: Firebase (Firestore, Auth, Storage)

Table of Contents

1. [What This App Does](#)
2. [How to Run the App](#)
3. [Technology Stack](#)
4. [App Architecture](#)
5. [Project Folder Structure](#)
6. [User Types & Roles](#)
7. [Authentication System](#)
8. [Complete Screen-by-Screen Guide](#)
9. [Data Models Explained](#)
10. [Backend Services](#)
11. [State Management \(Riverpod\)](#)
12. [Database Collections & Structure](#)
13. [Security Rules](#)
14. [Database Indexes](#)
15. [Localization \(English & Arabic\)](#)
16. [Theme & Design System](#)
17. [Navigation & Routing](#)
18. [Firebase Project Configuration](#)

19. [What's Production-Ready](#)

20. [What Still Needs External Setup](#)

1. What This App Does

DriverApp is a **subscription-based private transportation platform** designed for **Saudi Arabia**. Unlike Uber/Careem where you book individual rides, DriverApp works on a **monthly subscription model** for daily commuters.

The Problem It Solves

- People who commute daily (home to work, work to home) need to book two rides every day
- This is expensive with surge pricing and unpredictable
- No option to choose driver gender (important in Saudi Arabia)
- No consistency - different driver every time

How It Works

1. **User subscribes** to a monthly plan (Basic/Standard/Premium)
2. **Sets their schedule** (which days, pickup time, return time)
3. **Chooses preferences** (vehicle type, driver gender)
4. **Gets automatic daily rides** - driver assigned, trips created automatically
5. **Tracks rides in real-time** on a map
6. **Rates drivers** after each trip

Key Features

- Fixed monthly pricing (SAR currency)
- Pre-scheduled rides (set it once, rides happen daily)
- Driver gender selection (Male/Female/No Preference)
- Vehicle type choices (Economy/Comfort/Luxury/Van)
- Real-time GPS tracking
- In-app support chat
- Referral system (earn SAR 50 per friend)
- Admin dashboard for managing the platform

- Full Arabic + English support
-

2. How to Run the App

Prerequisites

- Flutter SDK 3.9.0+
- Dart SDK
- Android Studio or VS Code
- Firebase project configured

Steps

```
# 1. Get dependencies
flutter pub get

# 2. Generate localization files (if needed)
flutter gen-l10n

# 3. Run on device/emulator
flutter run

# 4. Build release APK
flutter build apk --release

# 5. Build iOS
flutter build ios --release

# 6. Deploy Firestore rules & indexes
firebase deploy --only firestore:rules,firestore:indexes
```

3. Technology Stack

Frontend (Mobile App)

Technology	Purpose	Why We Chose It
Flutter	Cross-platform framework	Single codebase for iOS + Android
Dart	Programming language	Type-safe, fast compilation
Riverpod	State management	Modern, testable, no context dependency
Google Maps Flutter	Maps & tracking	Industry standard for maps
Google Fonts (Poppins)	Typography	Clean, modern, supports Arabic

Backend (Firebase)

Service	Purpose	Why We Chose It
Firebase Auth	User authentication	Free email auth, easy integration
Cloud Firestore	Database	Real-time sync, offline support
Firebase Storage	File storage	Profile images, documents
Firebase Messaging	Push notifications	Reliable delivery

All Dependencies (from pubspec.yaml)

```
Core:
  flutter_riverpod: ^2.6.1      - State management
  firebase_core: ^3.8.1        - Firebase initialization
  firebase_auth: ^5.3.4        - Authentication
  cloud_firestore: ^5.6.0      - Database
  firebase_storage: ^12.3.7    - File storage
  firebase_messaging: ^15.1.6  - Push notifications

Maps & Location:
  google_maps_flutter: ^2.10.0 - Map display
  geolocator: ^13.0.2         - GPS location
  geocoding: ^3.0.0           - Address lookup

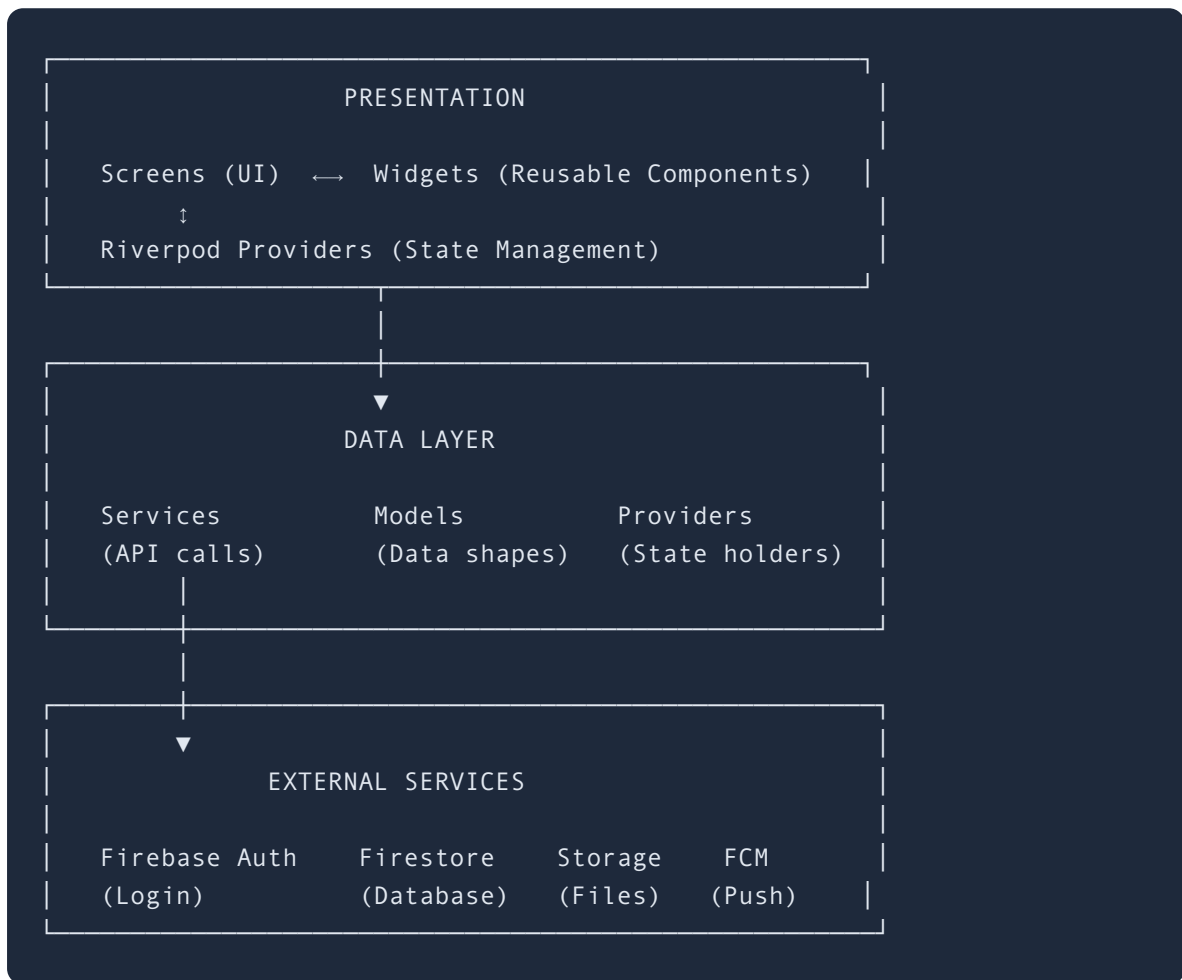
UI Components:
  google_fonts: ^6.2.1        - Custom fonts
  shimmer: ^3.0.0              - Loading skeletons
  pininput: ^5.0.1             - OTP input (legacy)
  cached_network_image: ^3.4.1 - Image caching
  flutter_svg: ^2.0.16         - SVG support
  image_picker: ^1.1.2         - Camera/gallery

Utilities:
  intl: ^0.20.1                - Date/number formatting
  shared_preferences: ^2.3.4    - Local settings storage
  flutter_secure_storage: ^9.2.3 - Encrypted storage
  url_launcher: ^6.3.1         - Open URLs/phone
  uuid: ^4.5.1                 - Generate unique IDs

Payments:
  flutter_stripe: ^11.3.0       - Card payments (future)
```

4. App Architecture

The app follows a **clean architecture** pattern with clear separation of concerns:



How Data Flows

1. **User taps a button** on a Screen
2. **Screen calls a Provider** (e.g., `ref.read(firestoreServiceProvider)`)
3. **Provider calls a Service method** (e.g., `firestoreService.cancelTrip(tripId)`)
4. **Service talks to Firebase** (reads/writes Firestore documents)
5. **Firebase sends updates back** via Streams (real-time)
6. **Provider notifies the Screen** to rebuild with new data
7. **Screen shows updated UI** to the user

5. Project Folder Structure

```

lib/
├─ main.dart                ← App entry point (initializes Firebase)
├─ app.dart                 ← Root widget (theme, locale, routes)
├─ firebase_options.dart    ← Auto-generated Firebase config
├─
├─ config/
│   └─ routes.dart          ← All 39 navigation routes defined here
├─
├─ core/                    ← Shared utilities used everywhere
│   └─ animations/
│       ├── fade_animation.dart    ← Fade-in effect
│       ├── staggered_animation.dart ← Sequenced animations
│       ├── pulse_animation.dart    ← Pulsing glow effect
│       └─ page_transitions.dart    ← Screen transition animations
│   ├── constants/
│       └─ app_constants.dart       ← Firebase collection names, limits
│   ├── enums/
│       ├── enums.dart              ← All app enumerations (10 enums)
│       └─ enum_l10n.dart           ← Localized enum display names
│   ├── theme/
│       ├── app_colors.dart         ← All color definitions
│       └─ app_theme.dart           ← Light & dark theme configs
│   └─ localization/
│       └─ locale_provider.dart     ← Language switching logic
├─
├─ data/                     ← Data layer (models, services, state)
│   ├── models/
│       ├── user_model.dart         ← User profile data shape
│       ├── trip_model.dart         ← Trip/ride data shape
│       ├── subscription_model.dart ← Subscription data shape
│       ├── plan_model.dart         ← Subscription plan data shape
│       ├── address_model.dart      ← Location/address data shape
│       └─ schedule_model.dart      ← Schedule data shape
│   ├── services/
│       ├── auth_service.dart       ← Authentication (email/password)
│       ├── firestore_service.dart  ← ALL database operations (60+ methods)
│       ├── cache_service.dart      ← Local caching
│       └─ connectivity_service.dart ← Network status
│   └─ providers/
│       └─ providers.dart           ← ALL Riverpod state providers (30+)
├─
├─ l10n/                     ← Localization (translations)
│   ├── app_en.arb               ← English strings (200+)
│   ├── app_ar.arb               ← Arabic strings (200+)
│   └─ generated/
│       ├── app_localizations.dart  ← Auto-generated from .arb files
│       ├── app_localizations_en.dart
│       └─ app_localizations_ar.dart
├─
└─ presentation/             ← UI layer (screens + widgets)

```


└─ screens/	
└─ auth/	← Login, profile setup (3 files)
└─ splash/	← Splash screen (1 file)
└─ onboarding/	← Onboarding slides (1 file)
└─ home/	← Main dashboard (1 file)
└─ subscription/	← Plans, schedule, payment (4 files)
└─ schedule/	← Calendar & schedule (1 file)
└─ history/	← Trip history & details (3 files)
└─ tracking/	← Real-time trip map (1 file)
└─ rating/	← Rate driver screen (1 file)
└─ profile/	← Profile management (4 files)
└─ settings/	← Settings & legal (4 files)
└─ notifications/	← Notification center (1 file)
└─ support/	← Help & live chat (2 files)
└─ offers/	← Promo codes (1 file)
└─ referral/	← Referral program (1 file)
└─ common/	← Shared screens (1 file)
└─ admin/	← Admin panel (5 files)
└─ driver/	← Driver panel (5 files)
└─ widgets/	
└─ home/	← Home screen cards (4 files)
└─ common/	← Reusable UI components (4 files)

Total: ~55 screen files, 8 widget files, 6 models, 4 services, 30+ providers

6. User Types & Roles

The app has **3 user roles**, each seeing different screens:

1. Passenger (Regular User)

- **Role value:** `user`
- **Home screen:** `/home`
- **What they can do:**
 - Browse and subscribe to plans
 - Set pickup/dropoff schedule
 - Track rides in real-time
 - Rate drivers
 - Manage profile, addresses, payment methods
 - Use referral codes
 - Chat with support

2. Driver

- **Role value:** `driver`
- **Home screen:** `/driver-home`
- **What they can do:**
 - Toggle online/offline status
 - View assigned trips
 - Accept/decline trip requests
 - Navigate to passenger (arriving status)
 - Start/complete trips
 - View earnings and stats
 - Register as driver (fill vehicle info)

3. Admin

- **Role value:** `admin`
- **Home screen:** `/admin`
- **What they can do:**
 - View platform statistics (users, trips, revenue)
 - Manage all users (activate/deactivate)
 - Manage all drivers (view status, stats)
 - Manage all trips (assign drivers, cancel)
 - View analytics (revenue by period, trip volume)

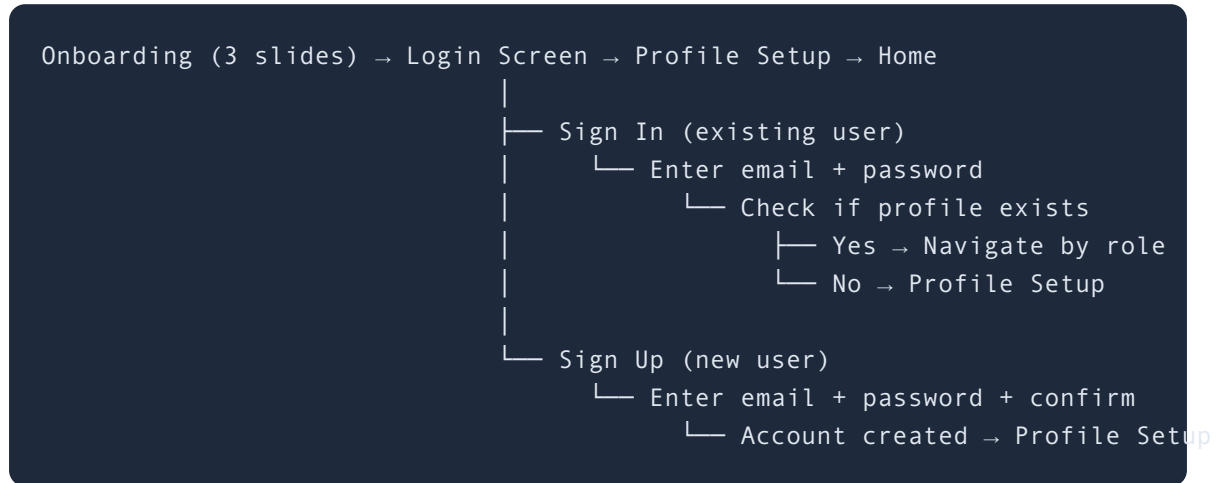
How Roles Are Determined

When a user logs in, the splash screen checks their `role` field in Firestore and navigates to the appropriate home screen:

```
Splash → Check auth → Check Firestore role → Navigate:  
- role = "user"    → /home  
- role = "driver"  → /driver-home  
- role = "admin"   → /admin  
- No Firestore doc → /profile-setup (new user)
```

7. Authentication System

Current Flow: Email + Password



Profile Setup (New Users)

After registration, new users complete a **2-step profile**:

Step 1 - Basic Info:

- Full name (required)
- Phone number (required, e.g., +966 5XX XXX XXXX)
- Profile photo (optional, uploaded to Firebase Storage)

Step 2 - Emergency Contact:

- Contact name (optional but recommended)
- Contact phone (optional but recommended)

Password Reset

Users can reset their password by entering their email on the login screen. Firebase sends a password reset link.

Where the Code Lives

File	What It Does
<code>lib/data/services/auth_service.dart</code>	All auth logic (sign in, register, reset, delete)
<code>lib/presentation/screens/auth/login_screen.dart</code>	Login/Register UI
<code>lib/presentation/screens/auth/profile_setup_screen.dart</code>	Profile completion form
<code>lib/presentation/screens/splash/splash_screen.dart</code>	Auth state check + role-based routing

Auth Methods Available in Code

```
// Email authentication
signInWithEmail(email, password)    // Login
registerWithEmail(email, password)  // Register
sendPasswordResetEmail(email)       // Forgot password

// User management
userExists(uid)                     // Check if profile exists
createUser(UserModel)               // Create profile in Firestore
getUserData(uid)                    // Fetch profile
updateUser(UserModel)               // Update profile
deleteAccount()                     // Delete account + data
signOut()                           // Logout

// Phone auth (available but not primary)
sendOTP(phoneNumber, callbacks)     // Send SMS code
verifyOTP(otp)                      // Verify SMS code
```

8. Complete Screen-by-Screen Guide

8.1 Splash Screen (`splash_screen.dart`)

Route: `/` (first screen)

What it does:

- Shows animated app logo with pulse effect

- Shows "DriverApp" text with slide animation
- Shows loading progress bar
- After 2.5 seconds, checks if user is logged in:
- Not logged in → Onboarding
- Logged in → Checks role in Firestore → Routes to correct home screen

8.2 Onboarding Screen (`onboarding_screen.dart`)

Route: `/onboarding`

What it does:

- 4 swipeable pages explaining the app:
 1. "Pre-scheduled rides" - How subscriptions work
 2. "Choose your driver" - Gender preference feature
 3. "Pick your ride" - Vehicle type selection
 4. "Subscribe and save" - Cost savings message
- "Skip" button and "Get Started" button
- Page indicators (dots)
- Navigates to Login screen

8.3 Login Screen (`login_screen.dart`)

Route: `/login`

What it does:

- Animated gradient background
- Toggle between **Sign In** and **Sign Up** modes
- **Sign In mode:** Email field + Password field + Forgot Password link
- **Sign Up mode:** Email + Password + Confirm Password
- Form validation (email format, password length ≥ 6 , passwords match)
- Error messages for common issues (wrong password, user not found, email in use)
- "Terms of Service" and "Privacy Policy" links at bottom
- On success: navigates based on whether profile exists

8.4 Profile Setup (`profile_setup_screen.dart`)

Route: `/profile-setup`

What it does:

- 2-step form with step indicators
- **Step 1:** Profile photo picker + Full name + Phone number
- **Step 2:** Emergency contact name + Emergency contact phone
- Uploads profile photo to Firebase Storage

- Creates user document in Firestore `users` collection
- Navigates to `/home` on completion

8.5 Home Screen (`home_screen.dart`)

Route: `/home`

What it does:

- **Greeting header** - "Good Morning, Ahmed!" with search bar
- **Next Trip Card** - Shows next scheduled ride with:
 - Pickup → Dropoff route
 - Time and date
 - Driver name, rating, vehicle
 - "Track" and "Cancel" buttons
- **Subscription Card** - Shows:
 - Plan name and status
 - Trips remaining (e.g., "26 of 40")
 - Progress bar
 - Renewal date
 - "Manage" button
- **Quick Actions** - Grid of shortcuts:
 - Schedule, History, Support, Referral, etc.
- Bottom navigation bar: Home, Schedule, Trips, Profile

8.6 Plans Screen (`plans_screen.dart`)

Route: `/plans`

What it does:

- Lists available subscription plans from Firestore
- Each plan card shows: name, price, features list, "Popular" badge
- **Vehicle type selector** - Economy (1.0x) / Comfort (1.3x) / Luxury (2.0x) / Van (1.5x)
- **Driver gender selector** - Male / Female / No Preference
- Price updates dynamically based on vehicle type multiplier
- "Subscribe" button → Payment Confirmation → Schedule Setup

8.7 Schedule Setup (`schedule_setup_screen.dart`)

Route: `/schedule-setup`

What it does:

- 4-step wizard:
 1. **Day Selection** - Pick which days (quick: Weekdays/All/Custom)
 2. **Time Selection** - Morning pickup time + optional return time

- 3. **Location Selection** - Pickup and dropoff locations
- 4. **Summary** - Review all selections
 - Trip count calculation (e.g., "5 days × 2 trips = 10 trips/week")

8.8 Schedule Screen (`schedule_screen.dart`)

Route: `/schedule`

What it does:

- **Calendar view** - Monthly calendar with selectable dates
- **Trip list** - Shows trips for selected date
- Each trip card shows: time, status badge, route (pickup → dropoff)
- **Modify trip** - Change pickup time (time picker)
- **Cancel trip** - Confirmation dialog → calls `cancelTrip()` in Firestore

8.9 Trip Tracking Screen (`trip_tracking_screen.dart`)

Route: `/trip-tracking`

What it does:

- **Full-screen Google Map** with:
 - Green marker = Pickup location
 - Red marker = Dropoff location
 - Blue marker = Driver's current position (real-time from Firestore)
 - Polyline route between points
- **Status badge** - Shows current trip status with pulse animation
- **Driver info card** - Name, rating, vehicle number, contact buttons
- **Route info** - Pickup and dropoff addresses
- **SOS button** - Emergency actions (call 911, alert contact, share location)
- **Share trip** - Send trip status via SMS/WhatsApp/Emergency contact
- **Cancel trip** - With confirmation dialog
- Real-time updates via Firestore stream listener

8.10 Trip History (`trip_history_screen.dart`)

Route: `/history`

What it does:

- Lists completed and cancelled trips from Firestore
- Grouped by month
- Each card shows: date, time, route, status, rating (if given)
- Tap → Trip Details screen

8.11 Trip Details (`trip_details_screen.dart`)

Route: `/trip-details`

What it does:

- Complete trip information:
- Status banner (completed/cancelled)
- Route card (pickup → dropoff)
- Time details (scheduled, actual pickup, actual dropoff)
- Driver info (name, rating, vehicle)
- Fare breakdown
- Rating display
- Action buttons: View Receipt, Rate Driver

8.12 Trip Receipt (`trip_receipt_screen.dart`)

Route: `/trip-receipt`

What it does:

- Formatted receipt display:
- Invoice number
- Trip ID, date, duration, distance
- Driver and vehicle info
- Fare breakdown (base fare, distance charge, time charge, service fee, discount)
- Payment method used
- Total amount in SAR

8.13 Rate Driver Screen (`rate_driver_screen.dart`)

Route: `/rate-driver`

What it does:

- Driver avatar and info
- **5-star rating** with animated stars
- **Tag selection** - Changes based on rating:
 - 4-5 stars: "Smooth driving", "On time", "Friendly", "Clean car"
 - 1-3 stars: "Late arrival", "Rude", "Reckless driving", "Dirty car"
- **Feedback text field** (200 character limit)
- Submits rating to Firestore via `rateTrip()` method

8.14 My Subscription (`my_subscription_screen.dart`)

Route: `/my-subscription`

What it does:

- Active subscription card with gradient
- Usage stats (trips used / total, progress bar)
- Schedule summary (active days, times, locations)
- Billing info (price, discount, next billing date)
- **Pause subscription** - Choose duration (3/7/14 days)
- **Cancel subscription** - Select reason → cancels in Firestore
- **Upgrade** - Navigate to plans screen

8.15 Profile Screen (`profile_screen.dart`)

Route: `/profile`

What it does:

- Profile photo and name
- Menu items linking to:
 - Personal Information
 - Saved Addresses
 - Payment Methods
 - My Subscription
 - Settings
 - Support
 - Referral
 - Logout

8.16 Personal Info (`personal_info_screen.dart`)

Route: `/personal-info`

What it does:

- View/edit profile with toggle edit mode
- Fields: Name, Email, Phone (read-only)
- Emergency contact fields
- Profile photo change (camera or gallery)
- **Saves to Firestore** via `updateUser()` + uploads photo to Storage

8.17 Saved Addresses (`saved_addresses_screen.dart`)

Route: `/saved-addresses`

What it does:

- List of saved addresses with type icons (Home/Work/School/Other)
- Default address badge
- **Add address** - Bottom sheet with: type selector, label, street, building, floor, apartment, landmark

- **Edit address** - Same form pre-filled
- **Delete address** - Confirmation dialog
- **Set as default** - One-tap
- All operations **persist to Firestore** via `updateSavedAddresses()`

8.18 Payment Methods (`payment_methods_screen.dart`)

Route: `/payment-methods`

What it does:

- Credit/debit cards displayed as styled card UI (Visa blue, Mastercard dark)
- Digital wallets section (Apple Pay)
- **Add card** - Form: card number (formatted), expiry, CVV, cardholder name
- **Set default** - Firestore update
- **Remove card** - Delete from Firestore subcollection
- Security note about encryption

8.19 Settings Screen (`settings_screen.dart`)

Route: `/settings`

What it does:

- **Preferences:** Theme (Light/Dark/Auto), Language (English/Arabic)
- **Notifications:** Toggle push, email, trip reminders (SharedPreferences)
- **Security:** Biometric login toggle
- **Links:** Login History, Terms, Privacy Policy
- **Account:** Clear Cache, Delete Account (calls `authService.deleteAccount()`)
- **Logout** (calls `authService.signOut()`)

8.20 Notifications (`notifications_screen.dart`)

Route: `/notifications`

What it does:

- Tab bar: All, Trips, Billing, Other
- Real-time notification list from Firestore `notifications` collection
- Unread badge indicator
- Swipe to delete
- Tap to mark as read
- "Read All" button
- Type-specific icons and colors

8.21 Live Chat (`live_chat_screen.dart`)

Route: `/live-chat`

What it does:

- Real-time chat with support using Firestore `chats` collection
- Message bubbles (user = blue right, agent = white left)
- Quick reply chips for common topics
- Messages persisted via `sendChatMessage()`
- Chat agent shows "Online" status

8.22 Support Screen (`support_screen.dart`)

Route: `/support`

What it does:

- FAQ sections with expandable answers
- Contact options: Email (mailto:), Phone (tel:), Report Problem
- Quick action tiles for common issues
- Link to Live Chat

8.23 Referral Screen (`referral_screen.dart`)

Route: `/referral`

What it does:

- Gradient header with "Invite Friends & Earn"
- Referral code display (auto-generated from Firestore)
- Copy to clipboard button
- Share buttons: SMS, WhatsApp, Email, More
- Stats: Friends referred, Credits earned, Per referral (SAR 50)
- How it works (3 steps)
- Referral history list (from Firestore `referrals` collection)

8.24 Promo Offers (`promo_offers_screen.dart`)

Route: `/promo-offers`

What it does:

- Enter promo code field
- **Validates via Firestore** `validatePromoCode()`
- Lists active promo codes from `promoCodes` collection
- Each offer shows: code, description, discount %, max discount, expiry
- Apply button

8.25 Location Picker (`location_picker_screen.dart`)

Route: `/location-picker`

What it does:

- Interactive Google Map with draggable center pin
- Search bar for finding locations
- Recent locations list
- Popular places (Riyadh locations)
- Returns selected location with coordinates

8.26-8.30 Admin Screens

Admin Dashboard (`admin_dashboard_screen.dart`)

Route: `/admin`

- **Real stats from Firestore:** Active users, Today's trips, Active subscriptions, Active drivers
- **Monthly revenue** calculated from completed trips
- Management navigation tiles (Users, Drivers, Trips, Analytics)
- Recent activity feed

User Management (`user_management_screen.dart`)

Route: `/admin/users`

- Real user list from `allUsersProvider`
- Search by name/email
- Filter: All / Active / Inactive
- Toggle user active/inactive (calls `toggleUserActive()`)

Driver Management (`driver_management_screen.dart`)

Route: `/admin/drivers`

- Real driver list from `allDriversProvider`
- Grouped: Online / Offline
- Shows: name, rating, total trips, vehicle info, status

Trip Management (`trip_management_screen.dart`)

Route: `/admin/trips`

- All trips from `allTripsProvider`
- Assign driver to unassigned trips (picks from `availableDriversProvider`)

- Cancel trips
- Status-based filtering

Analytics (`analytics_screen.dart`)

Route: `/admin/analytics`

- Period selector: Today / This Week / This Month / This Year
- Revenue chart (from Firestore completed trips)
- Trip volume bar chart (last 7 days)
- Subscription distribution (by plan)
- Top drivers (by trip count)
- Popular routes analysis

8.31-8.35 Driver Screens

Driver Home (`driver_home_screen.dart`)

Route: `/driver-home`

- Online/Offline/Busy status toggle
- Today's trip count and earnings
- Assigned trips list
- Quick stats (rating, acceptance rate, total trips)

Driver Trips (`driver_trips_screen.dart`)

Route: `/driver-trips`

- Assigned trips with accept/decline
- Active trip with navigation controls
- Trip history

Driver Earnings (`driver_earnings_screen.dart`)

Route: `/driver-earnings`

- Period-based earnings (Today/Week/Month)
- Total earnings, trip count, distance, average per trip
- Earnings history

Driver Profile (`driver_profile_screen.dart`)

Route: `/driver-profile`

- Driver info and vehicle details
- Rating and stats display

Driver Registration (`driver_registration_screen.dart`)

Route: `/driver-registration`

- Multi-step form: vehicle info, license, insurance
- Saves to Firestore via `registerAsDriver()`

9. Data Models Explained

UserModel - Who is this person?

```
UserModel {  
  id           → Firebase Auth UID (e.g., "abc123xyz")  
  name         → "Ahmed Mohammed"  
  email        → "ahmed@email.com"  
  phone        → "+966 55 123 4567"  
  profileImageUrl → URL to Firebase Storage image  
  emergencyContact → "+966 50 987 6543"  
  emergencyContactName → "Fatima Ahmed"  
  savedAddresses → List of home, work, etc.  
  preferredDriverGender → male / female / noPreference  
  preferredVehicleType → low / mid / luxury / van  
  role         → user / driver / admin  
  isVerified   → true/false  
  isActive     → true/false  
  createdAt    → When they signed up  
  updatedAt    → Last profile change  
  
  // Driver-only fields:  
  vehicleModel → "Toyota Camry 2024"  
  vehiclePlate → "ABC 1234"  
  vehicleColor → "White"  
  driverRating → 4.8  
  totalTrips   → 156  
  driverStatus → online / offline / onTrip / busy  
}
```

TripModel - A single ride

```
TripModel {  
  id          → Firestore document ID  
  userId      → The passenger  
  driverId    → The assigned driver  
  subscriptionId → Which subscription this trip belongs to  
  pickupLocation → Address with coordinates  
  dropoffLocation → Address with coordinates  
  scheduledTime → When the trip should happen  
  actualPickupTime → When driver actually picked up  
  actualDropoffTime → When trip actually ended  
  status      → scheduled → driverAssigned → driverArriving → inProgress → completed  
  vehicleType → Type of car  
  driverName  → "Ahmed Khan"  
  vehicleNumber → "ABC-1234"  
  rating      → 0-5 stars (after trip)  
  feedback    → "Great driver, clean car"  
  fare        → 45.0 SAR  
  distanceKm  → 12.3  
  estimatedMinutes → 30  
}
```

SubscriptionModel - Monthly plan

```
SubscriptionModel {  
  id          → Firestore document ID  
  userId      → Who subscribed  
  planId      → Which plan (Basic/Standard/Premium)  
  planName    → "Standard"  
  planType    → monthly  
  vehicleType → mid  
  driverGender → noPreference  
  status      → active / paused / cancelled / expired  
  startDate   → When subscription started  
  endDate     → When it expires  
  schedule    → Days, times, locations  
  totalTrips  → 40  
  usedTrips   → 14  
  basePrice   → 599.0 SAR  
  finalPrice  → 539.0 SAR (after discount)  
  promoCode   → "WELCOME25"  
  discount    → 60.0 SAR  
}
```

PlanModel - Available subscription plans

```
PlanModel {  
  id          → "plan-standard"  
  name        → "Standard"  
  description  → "Great for daily commuters"  
  type        → monthly  
  durationDays → 30  
  tripsPerDay  → 2  
  basePrice    → 599.0 SAR  
  features     → ["2 trips per day", "Mid-range vehicle", ...]  
  isPopular    → true (shows badge)  
  isActive     → true (visible to users)  
}
```

Price calculation:

```
Economy car: 599 × 1.0 = SAR 599  
Comfort car: 599 × 1.3 = SAR 779  
Luxury car:  599 × 2.0 = SAR 1,198  
Van:         599 × 1.5 = SAR 899
```

AddressModel - A saved location

```
AddressModel {  
  id          → Unique ID  
  title        → "Home"  
  type        → home / work / school / other  
  address      → "123 King Fahd Road, Riyadh"  
  buildingName → "Sunrise Apartments"  
  floor        → "5"  
  apartment    → "502"  
  landmark     → "Near Al Faisaliah Tower"  
  latitude     → 24.7136  
  longitude    → 46.6753  
  isDefault    → true/false  
}
```

10. Backend Services

AuthService (`auth_service.dart`)

Handles everything related to user login and accounts.

Method	What It Does
<code>signInWithEmail(email, password)</code>	Log in existing user
<code>registerWithEmail(email, password)</code>	Create new account
<code>sendPasswordResetEmail(email)</code>	Send password reset link
<code>userExists(uid)</code>	Check if user has a profile in Firestore
<code>getUserData(uid)</code>	Get user profile data
<code>createUser(UserModel)</code>	Save new user profile
<code>updateUser(UserModel)</code>	Update existing profile
<code>deleteAccount()</code>	Delete user data + auth account
<code>signOut()</code>	Log out user

FirestoreService (`firestore_service.dart`)

This is the **main backend service** with 60+ methods. It handles ALL database operations.

User Operations (5 methods)

Method	What It Does
<code>getUserStream(userId)</code>	Real-time user data stream
<code>updateUser(userId, data)</code>	Update profile fields
<code>uploadProfileImage(userId, file)</code>	Upload photo to Firebase Storage
<code>updateSavedAddresses(userId, addresses)</code>	Save address list
<code>toggleUserActive(userId, isActive)</code>	Admin: activate/deactivate user

Subscription Operations (8 methods)

Method	What It Does
<code>getActivePlans()</code>	Get all subscription plans
<code>createSubscription(model)</code>	Create new subscription
<code>getActiveSubscription(userId)</code>	Get current active subscription
<code>getActiveSubscriptionStream(userId)</code>	Real-time subscription updates
<code>getUserSubscriptions(userId)</code>	All user subscriptions
<code>pauseSubscription(id)</code>	Pause subscription
<code>resumeSubscription(id)</code>	Resume from pause
<code>cancelSubscription(id, reason)</code>	Cancel with reason

Trip Operations (10 methods)

Method	What It Does
<code>createTrip(model)</code>	Create a new trip
<code>getUpcomingTrips(userId)</code>	Stream of future trips
<code>getNextTrip(userId)</code>	Get the very next trip
<code>getTripHistory(userId)</code>	Stream of past trips
<code>getTripStream(tripId)</code>	Real-time single trip updates
<code>updateTrip(tripId, data)</code>	Modify trip details
<code>cancelTrip(tripId)</code>	Cancel a trip
<code>rateTrip(tripId, rating, feedback)</code>	Submit driver rating
<code>getTodayTrips(userId)</code>	Get today's trips

Driver Operations (17 methods)

Method	What It Does
<code>registerAsDriver(userId, data)</code>	Convert user to driver
<code>updateDriverStatus(driverId, status)</code>	Set online/offline/busy
<code>getDriverAssignedTrips(driverId)</code>	Trips waiting for this driver
<code>getDriverActiveTrip(driverId)</code>	Currently in-progress trip
<code>getDriverTodayTrips(driverId)</code>	Today's schedule
<code>getDriverTripHistory(driverId)</code>	Past completed trips
<code>acceptTrip(tripId, driverId)</code>	Accept a trip assignment
<code>declineTrip(tripId)</code>	Decline a trip
<code>startDriverArriving(tripId)</code>	Mark as on the way
<code>startTrip(tripId)</code>	Mark passenger picked up
<code>completeTrip(tripId, fare, distance)</code>	Finish trip
<code>updateDriverLocation(id, lat, lng)</code>	Update GPS position
<code>getDriverEarnings(id, start, end)</code>	Calculate earnings for period
<code>getDriverStats(driverId)</code>	Get performance stats
<code>getAvailableDrivers()</code>	Online drivers (for assignment)
<code>getAllDrivers()</code>	All drivers (admin)
<code>assignDriverToTrip(...)</code>	Admin assigns driver to trip

Admin Operations (3 methods)

Method	What It Does
<code>getAllUsers()</code>	Stream all passenger users
<code>getAllTrips()</code>	Stream all trips
<code>getAdminStats()</code>	Dashboard stats (users, trips, revenue, etc.)

Notification Operations (5 methods)

Method	What It Does
<code>createNotification(...)</code>	Send notification to user
<code>getUserNotifications(userId)</code>	Stream notifications
<code>markNotificationRead(id)</code>	Mark single as read
<code>markAllNotificationsRead(userId)</code>	Mark all as read
<code>deleteNotification(id)</code>	Delete notification

Chat Operations (3 methods)

Method	What It Does
<code>sendChatMessage(...)</code>	Send support message
<code>getChatMessages(chatId)</code>	Stream chat messages
<code>getOrCreateChat(userId)</code>	Get or create chat session

Payment Operations (6 methods)

Method	What It Does
<code>createPayment(data)</code>	Record a payment
<code>getUserPayments(userId)</code>	Payment history stream
<code>getPaymentMethods(userId)</code>	Saved cards
<code>addPaymentMethod(userId, method)</code>	Add new card
<code>deletePaymentMethod(userId, methodId)</code>	Remove card
<code>setDefaultPaymentMethod(userId, methodId)</code>	Set primary card

Promo & Referral (4 methods)

Method	What It Does
<code>validatePromoCode(code)</code>	Check if promo code is valid
<code>getActivePromoCodes()</code>	List available promos
<code>getUserReferralCode(userId)</code>	Get/generate referral code
<code>getReferralStats(userId)</code>	Referral metrics

11. State Management (Riverpod)

Riverpod is our state management solution. Think of providers as **live data sources** that screens listen to. When data changes in Firebase, providers automatically notify screens to update.

How It Works (Simple Explanation)

```
Firebase (database)
  ↓ sends data
Provider (state holder)
  ↓ notifies
Screen (UI) rebuilds automatically
```

All Providers

Service Providers (3)

```
authServiceProvider      → Gives access to AuthService
firestoreServiceProvider → Gives access to FirestoreService
cacheServiceProvider    → Gives access to CacheService
```

Auth Providers (2)

```
authStateProvider      → Stream: Is user logged in? (User? object)
currentUserProvider    → Current Firebase Auth user
```

User Providers (2)

```
userDataProvider(userId)    → Stream: Any user's profile data  
currentUserDataProvider    → Stream: Current user's profile data
```

Subscription Providers (3)

```
plansProvider              → Future: List of available plans  
activeSubscriptionProvider → Stream: Current active subscription  
userSubscriptionsProvider → Stream: All user subscriptions
```

Trip Providers (4)

```
upcomingTripsProvider      → Stream: Upcoming rides  
tripHistoryProvider        → Stream: Past rides  
tripStreamProvider(tripId) → Stream: Single trip real-time updates  
nextTripProvider           → Future: Next scheduled trip
```

Driver Providers (8)

```
driverStatusProvider       → State: Online/Offline status  
driverAssignedTripsProvider → Stream: Trips assigned to driver  
driverActiveTripProvider   → Stream: Current in-progress trip  
driverTodayTripsProvider   → Stream: Driver's today schedule  
driverTripHistoryProvider  → Stream: Driver's completed trips  
driverEarningsProvider(period) → Future: Earnings by period  
driverStatsProvider        → Future: Driver performance stats  
allDriversProvider         → Stream: All drivers (admin)  
availableDriversProvider   → Stream: Online drivers (admin)
```

Admin Providers (3)

```
adminStatsProvider         → Future: Dashboard statistics  
allUsersProvider           → Stream: All users  
allTripsProvider           → Stream: All trips
```

Feature Providers (5)

<code>notificationsProvider</code>	→ Stream: User's notifications
<code>userPaymentsProvider</code>	→ Stream: Payment history
<code>promoCodesProvider</code>	→ Future: Active promo codes
<code>chatIdProvider</code>	→ Future: Chat session ID
<code>chatMessagesProvider(id)</code>	→ Stream: Chat messages

App State Providers (3)

<code>isFirstLaunchProvider</code>	→ State: First time opening app?
<code>selectedPlanProvider</code>	→ State: Currently selected plan
<code>onboardingCompleteProvider</code>	→ State: Has seen onboarding?

12. Database Collections & Structure

Firestore Collections Overview

```
Firestore Database
|
├── users/                                ← User profiles (passengers + drivers + admins)
│   ├── {userId}/
│   │   ├── name, email, phone, role, ...
│   │   └── paymentMethods/             ← Subcollection
│   │       ├── {methodId}/
│   │       │   ├── type, lastFour, expiryDate, isDefault
│   │       └──
│   └──
├── plans/                                ← Subscription plans (managed by admin)
│   ├── {planId}/
│   │   ├── name, basePrice, durationDays, features, isActive
│   │   └──
│   └──
├── subscriptions/                        ← User subscriptions
│   ├── {subscriptionId}/
│   │   ├── userId, planId, status, schedule, totalTrips, usedTrips
│   │   └──
│   └──
├── trips/                                ← Individual rides
│   ├── {tripId}/
│   │   ├── userId, driverId, pickupLocation, status, rating, fare
│   │   └──
│   └──
├── notifications/                       ← Push notification records
│   ├── {notificationId}/
│   │   ├── userId, title, message, type, isRead, createdAt
│   │   └──
│   └──
├── payments/                            ← Payment transaction records
│   ├── {paymentId}/
│   │   ├── userId, amount, status, type, createdAt
│   │   └──
│   └──
├── chats/                               ← Support chat sessions
│   ├── {chatId}/
│   │   ├── userId, status, lastMessage
│   │   └── messages/                  ← Subcollection
│   │       ├── {messageId}/
│   │       │   ├── senderId, message, isUser, createdAt
│   │       └──
│   └──
├── promoCodes/                         ← Promotional discount codes
│   ├── {promoId}/
│   │   ├── code, discountPercent, maxDiscount, expiresAt, isActive
│   │   └──
│   └──
└── referrals/                          ← Referral tracking
    ├── {referralId}/
    │   ├── referrerId, referredName, status, createdAt
    │   └──
    └──
```

13. Security Rules

The Firestore security rules (`firestore.rules`) enforce who can read/write what data.

Key Principles

1. **All access requires authentication** - No anonymous access
2. **Users can only access their own data** - With exceptions for drivers/admins
3. **Admins have elevated access** - Can manage all data
4. **Drivers can read trip data** - For trips assigned to them
5. **Plans and promo codes are publicly readable** - But only admins can modify

Quick Reference

Collection	Who Can Read	Who Can Write
users	Owner, Drivers (for trip info), Admins	Owner (own profile), Admins
plans	Any logged-in user	Admins only
subscriptions	Owner, Admins	Owner (create/pause/cancel), Admins
trips	Owner, Assigned driver, Admins	Owner (cancel/rate), Driver (status), Admins
notifications	Owner only	Admins/Cloud Functions (create), Owner (mark read/delete)
payments	Owner, Admins	Owner (create), Admins
chats + messages	Owner, Admins	Owner (send), Admins (respond)
promoCodes	Any logged-in user	Admins only
referrals	Referrer, Admins	Any logged-in user (create)
paymentMethods	Owner only	Owner only

14. Database Indexes

Firestore requires **composite indexes** for queries that filter/sort on multiple fields. We have **25 indexes** deployed.

Why They Matter

Without indexes, queries like "get all trips for user X where status is 'completed' ordered by date" would fail. Each index tells Firestore how to efficiently look up this data.

Index Summary

- **Trips collection:** 12 indexes (most complex - queried by userId, driverId, status, time)
- **Users collection:** 4 indexes (queried by role, status, creation date)
- **Subscriptions:** 2 indexes (queried by userId + status)
- **Notifications:** 2 indexes (queried by userId + read status)
- **Other collections:** 5 indexes (payments, chats, promos, plans)

Deployed To

These are automatically deployed via:

```
firebase deploy --only firestore:indexes
```

15. Localization (English & Arabic)

The app fully supports **English** and **Arabic** with 200+ translated strings.

How It Works

- Translation files: `lib/l10n/app_en.arb` (English) and `lib/l10n/app_ar.arb` (Arabic)
- Flutter auto-generates typed accessor classes
- In code: `AppLocalizations.of(context)!.welcomeBack` → "Welcome Back" or "مرحبًا"
- User can switch language in Settings screen

- Language preference saved in SharedPreferences

What's Translated

- All screen titles and labels
- Button text
- Error messages
- Notification content
- Enum display names (trip statuses, vehicle types, etc.)
- Legal content (Terms, Privacy Policy)

16. Theme & Design System

Colors

Color	Hex	Usage
Primary Blue	#2563EB	Main actions, headers, links
Primary Dark	#1D4ED8	Gradients, app bars
Secondary Green	#10B981	Success states, online status
Error Red	#EF4444	Errors, cancel actions, SOS
Warning Amber	#F59E0B	Warnings, pending states
Background	#F8FAFC	Page backgrounds
Surface White	#FFFFFF	Cards, inputs
Text Primary	#1E293B	Main text
Text Secondary	#64748B	Subtitles, labels

Dark Theme

Full dark mode support with matching dark color palette:

- Background: #0F172A
- Surface: #1E293B
- Text: #F1F5F9

Typography

- **Font:** Google Fonts - Poppins
- **Sizes:** 10px (captions) to 42px (hero titles)
- **Weights:** Regular (400), Medium (500), SemiBold (600), Bold (700)

Component Styling

- **Border Radius:** 10-24px (rounded corners throughout)
- **Shadows:** Subtle elevation shadows for cards
- **Animations:** Fade, slide, scale, pulse effects on most screens

17. Navigation & Routing

39 Total Routes

All routes are defined in `lib/config/routes.dart` using Flutter's Navigator 1.0 with named routes.

Page Transitions

- **Fade** - Splash, Home (smooth entry)
- **Slide Right-to-Left** - Most screen pushes
- **Slide Bottom-to-Top** - Plans, Profile Setup (modal feel)
- **Scale** - Payment Confirmation (celebratory)

Navigation Flow

```
App Launch
├── Splash (2.5s animation)
│   └── Check Auth State
│       ├── Not Logged In → Onboarding → Login → Profile Setup → Home
│       └── Logged In → Check Role
│           ├── User → /home
│           ├── Driver → /driver-home
│           └── Admin → /admin
```

18. Firebase Project Configuration

Setting	Value
Project ID	drive-app-780b3
Project Number	93027596899
Region	Default
Auth Method	Email/Password (primary)
Database	Cloud Firestore
Storage	Firebase Storage
Platforms	Android, iOS, macOS, Web

19. What's Production-Ready

Feature	Status	Backend
Email/Password Auth	Ready	Firebase Auth
Profile Management	Ready	Firestore + Storage
Saved Addresses (CRUD)	Ready	Firestore
Subscription Plans	Ready	Firestore
Schedule Setup	Ready	Firestore
My Subscription (Pause/Cancel)	Ready	Firestore
Trip Tracking (Real-time)	Ready	Firestore Streams
Trip History	Ready	Firestore
Rate Driver	Ready	Firestore
Schedule (Cancel/Modify Trips)	Ready	Firestore
Notifications	Ready	Firestore
Live Chat	Ready	Firestore Real-time
Admin Dashboard (Stats)	Ready	Firestore Aggregations
Admin User Management	Ready	Firestore
Admin Driver Management	Ready	Firestore
Admin Trip Management	Ready	Firestore
Admin Analytics	Ready	Firestore Queries
Driver Home & Trips	Ready	Firestore Streams
Driver Earnings	Ready	Firestore
Driver Registration	Ready	Firestore
Payment Methods (CRUD)	Ready	Firestore Subcollection
Promo Codes	Ready	Firestore
Referral System	Ready	Firestore
Localization (EN/AR)	Ready	ARB Files
Dark Mode	Ready	ThemeData
Firestore Security Rules	Ready	Deployed
Firestore Indexes (25)	Ready	Deployed

20. What Still Needs External Setup

These require configuration outside the codebase:

Item	What's Needed	Difficulty
Stripe Payment	Add Stripe keys, complete payment flow for real charges	Medium
FCM Push Notifications	Configure in Firebase Console, add Cloud Functions for triggers	Medium
Google Places API	Get API key for location autocomplete search	Easy
Google Maps API Key	Already configured for map display, verify billing	Easy
Cloud Functions	Driver matching algorithm, auto-trip creation from schedule	Hard
App Store Submission	Screenshots, descriptions, review compliance	Medium

End of Documentation

This document covers every aspect of the DriverApp codebase. For questions about specific files or features, reference the file paths mentioned in each section.